

PyTesseract

December 29, 2025

Contents

1	Introduction	2
2	Page Segmentation Modes	2
3	Segmentation and Classification	2
3.1	Traditional PyTesseract Model	3
3.2	Modern PyTesseract Model	3
4	Dictionary Based Cleanup	4
4.1	Traditional System	4
4.2	Modern System	4
5	Concluding Remarks	5

1 Introduction

In the previous week, we worked with PyTesseract as a black-box OCR system, where our main focus was on image pre-processing techniques such as resizing, thresholding, noise removal, and contrast enhancement before feeding the image into the OCR engine. This week, we change the approach slightly and look inside PyTesseract to understand what actually happens after an image is given as input. Instead of treating it as a black box, we will break the OCR pipeline into clear stages—page segmentation, character recognition, character classification, and dictionary-based cleanup—to understand why certain pre-processing steps and PSM (Page Segmentation Modes) choices work better than others.

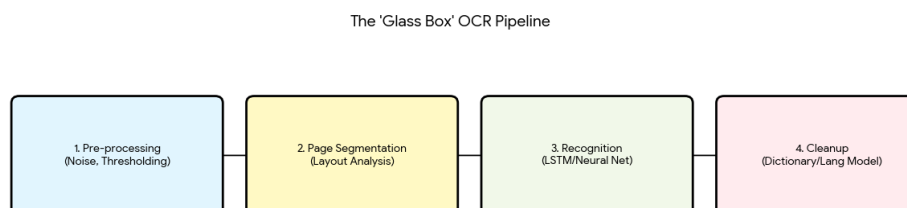


Figure 1: Pipeline of a PyTesseract Model

2 Page Segmentation Modes

Page Segmentation Modes (PSMs) tell Tesseract how to interpret the layout of the text in an input image before performing OCR. In other words, a PSM defines the assumptions the engine should make about the structure of the text — for example, whether the image contains a full page of text, a single line, a single word, or even just a single character. Choosing the right PSM can dramatically improve OCR accuracy, because if Tesseract assumes the wrong structure, it may return an empty result or nonsensical text even if the image is clean. You can read about the different PSMs [here](#).

3 Segmentation and Classification

After the Page Segmentation Mode (PSM) is selected, Tesseract uses it to decide how aggressively and at what structural level the image should be broken down. The PSM defines the assumed layout of the text present in the image—whether it should be treated as a full page containing multiple text blocks, a single block of text, a single line, a single word, or a single character. Based on this assumption, Tesseract performs layout analysis accordingly. The image is first binarized, if it is not already binarized, so that text pixels are clearly separated from the background. Once binarization is complete, Tesseract performs connected component analysis, where it scans the image to identify groups of pixels that are connected to each other. Each connected component represents a visually coherent region, which may correspond to a part of a character or sometimes an entire character.

These connected components are then grouped using geometric and spatial heuristics. This grouping is based on properties such as the relative distance between components, their alignment along a common baseline, their height similarity, and their overall spatial consistency. For example, components that are close together horizontally and have similar heights are

more likely to belong to the same word, while components aligned along a straight baseline are grouped into a line of text. Importantly, the way this grouping happens is influenced by the chosen PSM. A PSM configured for full-page text will allow grouping into multiple blocks and lines, whereas a single-line or single-word PSM will restrict grouping to a much smaller structural unit. At the end of this stage, Tesseract has identified text blocks, lines, and rough character-sized regions called blobs, but it still does not know the actual character identities.

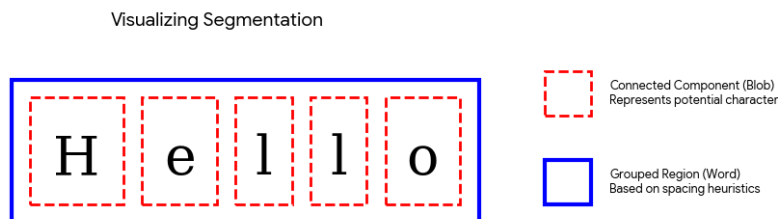


Figure 2: Segmentation from blobs to words

3.1 Traditional PyTesseract Model

In traditional OCR systems, once this segmentation was complete, recognition was performed at the individual character level. Each blob was assumed to correspond to a single character and was processed independently. From each isolated character image, visual features were extracted using manually designed filters and rules. These features included edges, contours, stroke thickness, pixel density, and shape descriptors. The filters used to extract these features were explicitly engineered by humans to capture distinguishing properties of characters in a compact numerical form. Once extracted, these feature vectors were compared against stored templates or passed through classical machine learning classifiers such as k-nearest neighbors or other simple models that had been trained on predefined character features. The classifier would then assign the most likely character label to each blob. While this approach worked for clean, well-separated printed text, it failed in cases where characters touched, handwriting was cursive, or characters were visually similar, such as ‘O’ and ‘0’ or ‘l’ and ‘1’, because each character was classified in isolation without any contextual information.

3.2 Modern PyTesseract Model

Modern versions of Tesseract no longer follow this segmentation-first, character-by-character recognition pipeline. Instead, after segmentation, the system treats recognition as a sequence modeling problem. Depending on the PSM, the image units passed to the recognizer are entire text lines, words, or larger text regions. For example, if the PSM assumes line-level text, each full line is extracted as a single image; if a larger structural PSM is used, multiple lines may be processed sequentially. These extracted image units are then fed into an LSTM-based neural network. In this modern pipeline, there is no strict separation between character recognition and classification. The LSTM processes the image from left to right, learning visual features such as strokes, shapes, and spacing patterns directly from data. Feature extraction and classification happen together inside the network, with the model jointly learning how to identify character boundaries and assign character labels. By modeling the entire sequence at once, the system

can use surrounding context to improve recognition, implicitly learning where characters begin and end and significantly improving accuracy on complex and noisy text.

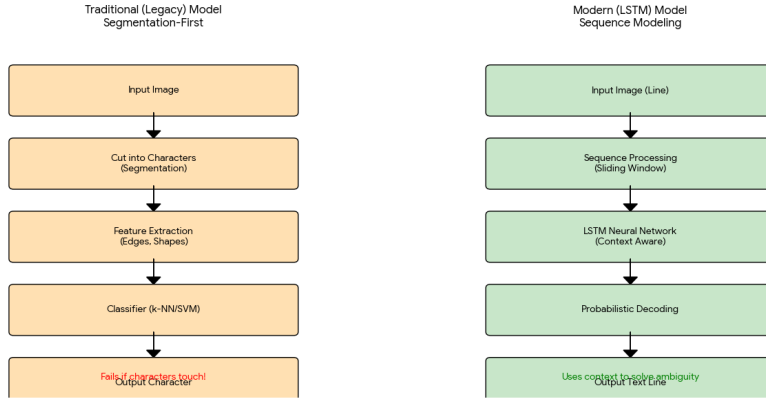


Figure 3: Traditional vs Modern classification

4 Dictionary Based Cleanup

Dictionary / Language Cleanup is the post-processing stage of an OCR pipeline where recognized text is refined using linguistic rules or learned language models to correct errors, enforce valid character sequences, and improve overall textual consistency.

4.1 Traditional System

In traditional OCR systems, the dictionary or post-processing stage was applied after character-level recognition and heuristic word formation had already taken place. Once characters were classified independently and grouped into words based on spacing rules, each resulting word was checked against a predefined dictionary. If the word existed in the dictionary, it was accepted as correct. If it did not, the system attempted to find the closest matching word using simple similarity measures such as edit distance. This approach helped correct obvious spelling errors but had several limitations. Because recognition was performed character by character without context, many errors originated early in the pipeline and could not be reliably fixed later. Additionally, fixed dictionaries struggled with proper nouns, technical terms, abbreviations, and domain-specific vocabulary, often leading to incorrect replacements.

4.2 Modern System

Modern OCR systems handle dictionary and language cleanup in a more context-aware manner by using learned language models instead of relying solely on static word lists. A learned language model captures statistical patterns of a language, such as which character sequences form valid words and which word sequences are likely to occur together. Rather than simply checking whether a word exists in a dictionary, the model assigns probabilities to different character and word sequences and evaluates how likely a given recognition output is within its surrounding context. This allows the system to prefer corrections that make linguistic sense in the given sequence, rather than blindly replacing words based on minimal spelling distance. Because modern OCR systems already incorporate contextual information during recognition through LSTM-based sequence models, the language cleanup stage works in close coordination with the recognition output. The cleanup process refines predictions by reinforcing contextually

consistent sequences rather than correcting isolated words. As a result, the system can handle ambiguous characters, unseen words, and complex text more effectively. This integration of sequence-based recognition and probabilistic language modeling makes modern OCR pipelines significantly more robust and accurate than traditional dictionary-based approaches

5 Concluding Remarks

So far, we have understood the complete OCR pipeline using PyTesseract from start to finish. We begin with image pre-processing to improve text clarity and reduce noise before feeding the image into the PyTesseract (Python wrapper) model. Once the image is passed into the model, the system internally performs page segmentation to decide how the text should be structured, followed by grouping text into lines and regions. After this, character recognition and classification are carried out using the underlying OCR model, and finally, dictionary or language-based cleanup is applied internally to refine the recognized text. This entire process produces the final textual output.

After obtaining the OCR output, we can evaluate its quality using error measures such as incorrect characters, missed words, or overall accuracy. Based on these results, we can improve performance by tuning internal parameters of the PyTesseract model. OCR systems expose configurable parameters—such as page segmentation modes, OCR engine modes, and language or configuration flags—that influence recognition behavior. By adjusting these parameters and observing their effect on the output, we can iteratively optimize the OCR system for better accuracy on a given dataset.